

PyQT5 vs PyQT6:

A comparison: <https://www.pythonguis.com/faq/pyqt5-vs-pyqt6/>

The consensus is that PyQT6 should be used because it's just more modern. It's built on the QT6 library for C++, rather than QT5. And I feel like modern=less chance for memory leak with cpp, which idk is safer idk

PyQT Research idk:

Info mostly from: <https://www.pythonguis.com/>

★ Main Modules:

- QtWidgets
 - [QMainWindow](#)
 - [QApplication](#)
 - [QWidget](#)
 - [QPushButton](#)
 - QSplitter
 - QDockWidget
 - QGridLayout
- QtGui
- QtCore

QtWidgets:

★ In QT, all top-level widget objects are windows

★ QMainWindow (Class)

- Handy main window module
- For custom windows, it's good to make a subclass of this one

- setCentralWidget (method)
 - Pass the widget that goes in the middle of the main window as argument

★ QApplication (class)

- Each application is an instance of class **QApplication** from PyQt6
 - From now on, I will call our instance of QApplication “**app**”
- To enable CLI:
 - Import lib: **sys**
 - Pass **sys.argv*** into the instance of **QApplication** made on initialisation:
 - `app = QApplication(sys.argv)`
- If CLI isn't required, pass an empty list in place of sys.argv
- To start the event loop, call QApplication member function “**exec**”
 - `app.exec()`
 - Any code following the above line will not be processed until the app is terminated

★ QWidget (Class)

- A QWidget object can act as a window in the GUI
 - `window = QWidget()`
- Windows are hidden by default. To have it be visible:
 - `window.show()`

★ QPushButton (Class)

- Makes a funny button idk what to tell you
- Can pass string on init, string will be printed on the button

★ setFixedSize (method)

- It's in the name
- Takes argument of type **QSize** from QtCore module

★ QSplitter

- Container that holds 2+ widgets with a draggable divider and users can resize sections live.
- Applicability: big main feed on the left, thumbnails list in the middle, and controls area in the right, users can drag/resize them
 - Ex: `splitter = QSplitter(Qt.Orientation.Horizontal)`
- If you use `QDockWidget` for controls, the `QSplitter` might only be the main feed vs thumbnails.

★ QDockWidget

- Right sidebar better than GTK Revealers currently
- A panel that can be docked right/left/bottom, collapsed, floated as separate window, toggled from menu/toolbar
- Can do docking + hiding
- Initiate `QWidget`:
 - `dock = QDockWidget("Controls", self)`
`dock.setAllowedAreas(`
`dock.DockWidgetArea.LeftDockWidgetArea |`
`dock.DockWidgetArea.RightDockWidgetArea`
`)`
 - `controls = QWidget()`
 - `dock.setWidget(controls)`
- To initiate a panel as a dock widget:
 - `self.addDockWidget(dock.DockWidgetArea.RightDockWidgetArea, dock)`
 - `dock.toggleViewAction()`

★ QVBoxLayout/QGridLayout

- Pattern A (Thumbnails in a vertical list): `QVBoxLayout`
 - Initialize `QVBoxLayout`:
 - `thumbs_widget = QWidget()`
 - `thumbs_layout = QVBoxLayout(thumbs_widget)`
 - Implement it within the cams:
 - for `i` in `range(8)`:
`thumbs_layout.addWidget(VideoTile(name=f"Cam {i+1}"))`
 - `thumbs_layout.addStretch()`
- Pattern B (Thumbnails in a grid like 2x4 or 3x3): `QGridLayout`
 - Initialize `QGridLayout`:
 - `grid_widget = QWidget()`
 - `grid = QGridLayout(grid_widget)`
 - Implement it within the cams:

- `idx = 0`
- `for r in range(2):`
 - `for c in range(4):`
 - `idx += 1`
 - `grid.addWidget(VideoTile(name=f"Cam {idx}"), r, c)`

GStreamer functionality in PyQt:

★ How to Render GStreamer video pipeline inside PyQt:

- Option A (for best performance and closest in to current design): Turning VideoOverlay into a QWidget
 - Using a normal sink like "glimagesink"
 - Creating a QWidget placeholder and passing its winId() to the sink via `GstVideoOverlay.set_window_handle(...)`
 - Close to `gtk sink.get_property("widget")` in the older code
 - However Qt doesn't give a sink widget, you give the sink your widget handle instead
 - Pipeline 1 (Software Decode which is most portable):
 - `srtsrc uri="srt://127.0.0.1:5001?latency=100"`
 - `wait-for-connection=true`
 - `! queue`
 - `! tsdemux latency=100`
 - `! h264parse config-interval=1`
 - `! avdec_h264`
 - `! videoconvert`
 - `! glimagesink name=videosink sync=false`
 - Pipeline 2 (Linux X11 sink variant with "ximagesink") if you are on Linux or X11 and want a simpler sink:
 - `srtsrc uri="srt://127.0.0.1:5001?latency=100"`
 - `wait-for-connection=true`
 - `! queue`
 - `! tsdemux latency=100`
 - `! h264parse config-interval=1`
 - `! avdec_h264`
 - `! videoconvert`
 - `! ximagesink name=videosink sync=false`
 - Pipeline 3 (Letting GStreamer decide decode with "decodebin"):
 - `srtsrc uri="srt://127.0.0.1:5001?latency=100"`
 - `wait-for-connection=true`
 - `! queue`

- !tsdemux
 - !queue
 - !decodebin
 - !videoconvert
 - !glimagesink name=videosink sync=false
- Option B (for more flexible uses like for overlays/recording/thumbnails like science cam): “appsink -> QImage -> QLabel/QOpenGLWidget”
 - GStreamer outputs decoded frames into “appsink”
 - You convert frames to numpy/QImage and paint them in Qt
 - Easier to implement custom overlays like scale indicator, crosshair, and recording badge but uses more CPU
 - Pipeline 1 (RGB frames to appsink which is easier to change):
 - srtsrc uri="srt://127.0.0.1:5001?latency=100" wait-for-connection=true
 - !queue
 - !tsdemux latency=100
 - !h264parse config-interval=1
 - !avdec_h264
 - !videoconvert
 - !video/x-raw,format=RGB
 - !appsink name=appsink emit-signals=true sync=false max-buffers=1 drop=true
 - Pipeline 2 (BGR frames which are better usually if we already use OpenCV):
 - ... (whatever before)
 - !videoconvert
 - !video/x-raw,format=BGR
 - !appsink name=appsink emit-signals=true sync=false max-buffers=1 drop=true
- For more immersive and dynamic cams option B better imo, but for 9 HD Streams option A still better.

★ Recreating the current feed title aspect in Qt

- Currently:
 - aspect ratio wrapper (Gtk.AspectFrame)
 - overlay container (Gtk.Overlay)
 - name label + fullscreen button + enable button
 - click to swap main feed with selected feed
- PyQt Equivalence:

- Aspect ratio: make a “QWidget” that overrides `resizeEvent` to maintain 16:9, or use “QLabel”/”QOpenGLWidget” inside a “QLayout” with spacers.
- Overlays: easiest is a “QStackedLayout” or just absolute-positioned children inside the tile widget.
- Swap main feed: just remove/insert widgets in layouts (Qt allows this cleanly), same concept as your GTK swap logic

★ Creating more immersive/dynamic look

- Layouts:
 - QSplitter: You are able to drag/resize main feed, thumbnails, and sidebar
 - QDockWidget: Sidebar with controls can dock/undock, collapse, and move sides better than the GTK revealer
- State Driven UI:
 - Create a small “app state” dictionary like object and update UI via signals
 - Qt’s signals/slots make the “settings changes -> update UI -> send TCP command” really clean instead of how we do it manually in the code.
 - Need to look and research more into this tho
- Non-blocking:
 - Using “QThread” or “QRunnable”/”QThreadPool” for the capture/stitch/SSH which we are currently using
“threading.Thread(...)” for background tasks
 - We can use “QTimer” for periodic UI refresh (FPS indicator or status polling)
- Styling:
 - Qt Stylesheets (`setStyleSheet`) = CSS-like styling across widgets.
 - QPropertyAnimation for smooth slide-in panels, fades, button hover expansions, etc.